



Universidade Federal Fluminense
Disciplina: Algoritmos em Grafos
Professor: Luís Felipe

Lista de revisão P1

1. *Problema - Dominó.* Há n peças do dominó, numeradas de 1 a n . Algumas peças estão posicionadas de modo que, se uma peça u cair, ela derruba diretamente a peça v . Essas relações são dadas por pares ordenados (u, v) .

Considere o dígrafo $G = (V, E)$ em que:

- cada vértice representa uma peça do dominó;
- existe uma aresta direcionada (u, v) se a queda da peça u provoca diretamente a queda da peça v .

Uma peça derrubada manualmente pode iniciar uma sequência de quedas, e todas as peças alcançáveis a partir dela no dígrafo também cairão.

Determine um conjunto mínimo de peças que devem ser derrubadas manualmente para que, ao final do processo, todas as peças caiam.

Considere a seguinte formatação da entrada e saída:

Entrada:

- A primeira linha contém dois inteiros n e m , o número de peças e o número de relações;
- cada uma das m linhas seguintes contém dois inteiros u e v , indicando que, se a peça u cair, então a peça v também cairá.

Saída: Saída: Imprima, em ordem crescente, uma lista de peças que forme um conjunto mínimo de peças a serem derrubadas manualmente para que todas as peças caiam. Em caso de múltiplas soluções mínimas, considere a solução obtida escolhendo a menor peça de cada CFC-fonte do grafo condensado.

Solução: Se uma peça u cai, então caem todas as peças alcançáveis a partir dela.

Agora, considere o *grafo condensado* do dígrafo, definido da seguinte forma:

- cada componente fortemente conexa (CFC) é contraída em um único vértice;
- para cada aresta (u, v) do grafo original, se u e v pertencem a CFCs distintas, adiciona-se uma aresta entre essas CFCs;
- o grafo resultante é um dígrafo acíclico (DAG).

Nesse DAG:

- se uma CFC A alcança outra CFC B , então ao derrubar uma peça de A , todas as peças de B também cairão;
- portanto, não é necessário escolher uma peça em cada CFC;
- é suficiente escolher uma peça em cada CFC-fonte do grafo condensado, isto é, cada CFC que não possui arestas de entrada vindas de outras CFCs.

Essas são exatamente as CFCs que não podem ser alcançadas a partir de nenhuma outra e, portanto, precisam ser iniciadas manualmente.

Assim:

i) Fazemos uma DFS no grafo original e calculamos, para cada vértice v :

- $PE[v]$: tempo de entrada de v ;
- $PS[v]$: tempo de saída de v ;
- $old[v]$: menor tempo de entrada de um vértice na mesma CFC de v que pode ser alcançado a partir do subgrafo enraizado em v , usando arestas da árvore DFS e no máximo uma aresta de retorno ou de cruzamento;
- $pai[v]$: pai de v na floresta DFS.

ii) A recorrência para $old(v)$ é:

$$old(v) = \min \begin{cases} PE(v), \\ \min\{old(w) \mid w \text{ é filho de } v \text{ em } T\}, \\ \min\{PE(w) \mid vw \text{ é aresta de retorno}\}, \\ \min\{PE(w) \mid vw \text{ é aresta de cruzamento, } v, w \text{ estão na mesma cfc}\} \end{cases}$$

iii) Na implementação, ao examinar uma aresta (v, w) :

- se w ainda não foi visitado, então (v, w) é aresta de árvore;
- se w foi descoberto, mas ainda não terminou, então (v, w) é aresta de retorno;
- se w já terminou e $PE[v] < PE[w]$, então (v, w) é aresta de avanço;
- se w já terminou e $PE[v] > PE[w]$, então (v, w) é aresta de cruzamento; nesse caso, ela só contribui para $old[v]$ se w ainda estiver na pilha.

iv) Mantemos uma pilha Q com os vértices descobertos e ainda não atribuídos a nenhuma CFC. Quando, ao final da exploração de v , tivermos

$$old[v] = PE[v],$$

então v é a raiz de uma CFC. Nesse caso, desempilhamos vértices até remover v ; todos os vértices removidos nesse processo pertencem à mesma CFC.

v) Após identificar todas as CFCs, percorremos todas as arestas do grafo original:

- se uma aresta (u, v) liga vértices de CFCs distintas, então ela representa uma aresta no grafo condensado;

- com isso, calculamos o grau de entrada de cada CFC no grafo condensado.
- vi) As CFCs com grau de entrada zero são exatamente aquelas que não podem ser alcançadas a partir de nenhuma outra CFC e, portanto, precisam ser iniciadas manualmente.
- vii) Para obter uma resposta única, escolhemos a menor peça de cada CFC-fonte e imprimimos os valores em ordem crescente.

```

1 def dfs_cfc(v, adj, pe, ps, old, pai, t, Q, emQ, cfc):
2     t[0] += 1
3     pe[v] = t[0]
4     old[v] = pe[v]          # inicializacao de old(v)
5     Q.append(v)            # empilha v
6     emQ[v] = True
7
8     for w in adj[v]:
9         if pe[w] == 0:      # w ainda nao foi visitado
10            pai[w] = v      # vw eh aresta da floresta DFS
11            dfs_cfc(w, adj, pe, ps, old, pai, t, Q, emQ, cfc)
12            old[v] = min(old[v], old[w])
13
14        elif ps[w] == 0:    # w foi descoberto, mas ainda nao terminou
15            old[v] = min(old[v], pe[w]) # vw eh aresta de retorno
16
17        else:              # w ja terminou
18            if pe[v] < pe[w]:
19                pass        # vw eh aresta de avanco
20            else:           # vw eh aresta de cruzamento
21                if emQ[w]:   # w ainda esta na pilha
22                    old[v] = min(old[v], pe[w])
23
24    t[0] += 1
25    ps[v] = t[0]
26
27    # v eh raiz de uma CFC
28    if old[v] == pe[v]:
29        comp = []
30        while True:
31            u = Q.pop()
32            emQ[u] = False
33            comp.append(u)
34            if u == v:
35                break
36        cfc.append(comp)
37
38
39 def solve():
40     n, m = map(int, input().split())
41
42     adj = [[] for _ in range(n + 1)]
43
44     for _ in range(m):
45         u, v = map(int, input().split())
46         adj[u].append(v)
47
48     # Apenas para deixar a execucao deterministica
49     for v in range(1, n + 1):
50         adj[v].sort()
51
52     pe = [0] * (n + 1)
53     ps = [0] * (n + 1)
54     old = [0] * (n + 1)
55     pai = [-1] * (n + 1)
56
57     t = [0]

```

```

58 Q = []
59 emQ = [False] * (n + 1)
60 cfc = []
61
62 # -----
63 # 1) Encontrar as CFCs com uma unica DFS usando old[v]
64 # -----
65 for v in range(1, n + 1):
66     if pe[v] == 0:
67         dfs_cfc(v, adj, pe, ps, old, pai, t, Q, emQ, cfc)
68
69 # -----
70 # 2) Identificar a CFC de cada vertice
71 # -----
72 comp_id = [-1] * (n + 1)
73
74 for i in range(len(cfc)):
75     for v in cfc[i]:
76         comp_id[v] = i
77
78 # -----
79 # 3) Menor vertice de cada CFC
80 # -----
81 menor_da_comp = [float('inf')] * len(cfc)
82
83 for v in range(1, n + 1):
84     c = comp_id[v]
85     if v < menor_da_comp[c]:
86         menor_da_comp[c] = v
87
88 # -----
89 # 4) Grau de entrada das CFCs no grafo condensado
90 # -----
91 indeg = [0] * len(cfc)
92
93 for u in range(1, n + 1):
94     for v in adj[u]:
95         if comp_id[u] != comp_id[v]:
96             indeg[comp_id[v]] += 1
97
98 # -----
99 # 5) Resposta: uma peça de cada CFC-fonte
100 # -----
101 resposta = []
102
103 for c in range(len(cfc)):
104     if indeg[c] == 0:
105         resposta.append(menor_da_comp[c])
106
107 resposta.sort()
108 print(*resposta)
109
110
111 solve()

```

2. Problema - Dominó II:

Considere o mesmo modelo do problema anterior (questão 1). Uma peça pode ser derrubada de duas formas:

- manualmente, iniciando uma nova sequência de quedas;
- automaticamente, ao ser alcançada a partir de outra peça no dígrafo correspondente.

O processo de derrubada ocorre em tempo discreto $t = 1, 2, 3, \dots$, conforme as regras:

- i) No instante $t = 1$, derruba-se manualmente a menor peça (em número) que ainda não foi derrubada;
- ii) A partir de uma peça derrubada manualmente, a propagação ocorre em camadas:
 - no instante seguinte, são derrubadas todas as peças diretamente alcançáveis a partir dela;
 - de forma geral, peças a uma aresta de distância são derrubadas antes das peças a duas arestas, e assim sucessivamente;
- iii) Quando não há mais peças alcançáveis a partir da sequência atual, inicia-se uma nova sequência:
 - escolhe-se a menor peça ainda não derrubada;
 - essa peça é derrubada no próximo instante de tempo disponível;
- iv) O processo continua até que todas as peças tenham sido derrubadas.

Durante a propagação, sempre que uma peça possui mais de um vizinho ainda não derrubado, estes devem ser considerados em ordem crescente de seus rótulos.

Tarefa: Dada uma peça x , determine:

- o instante de tempo em que a peça x será derrubada;
- um caminho no dígrafo, correspondente à árvore de propagação construída pelo processo acima, desde a peça que iniciou a sequência que derruba x até a peça x .

Entrada:

- A primeira linha contém dois inteiros n e m , o número de peças e o número de relações;
- Cada uma das m linhas seguintes contém dois inteiros u e v , indicando que, se a peça u cair, então a peça v também cairá;
- A última linha contém um inteiro x , a peça de interesse.

Saída:

- um inteiro correspondente ao instante de tempo em que a peça x será derrubada;
- na linha seguinte, uma sequência de vértices representando o caminho descrito acima;
- na última linha, um inteiro correspondente ao instante de tempo em que todas as peças terão sido derrubadas.

Solução: O processo descrito no enunciado corresponde a uma sequência de buscas em largura.

De fato:

- cada vez que uma peça é derrubada manualmente, inicia-se uma nova sequência de propagação;
- como as peças diretamente alcançáveis caem no instante seguinte, as peças a distância 2 caem depois, e assim sucessivamente, a propagação ocorre em camadas;
- portanto, para cada peça derrubada manualmente, a propagação deve ser simulada por uma BFS;
- quando essa BFS termina, escolhe-se a menor peça ainda não derrubada e inicia-se uma nova BFS.

Assim, o algoritmo consiste em percorrer os vértices em ordem crescente e, sempre que encontrarmos uma peça ainda não derrubada, iniciamos uma nova BFS a partir dela.

Durante a execução:

- $L[v]$ indica a ordem em que a peça v foi descoberta no processo;
- $pai[v]$ armazena a peça a partir da qual v foi derrubada na árvore de propagação;
- $nivel[v]$ indica a distância, em número de arestas, entre a raiz da BFS atual e o vértice v ;
- $tempo[v]$ armazena o instante de tempo em que a peça v foi derrubada.

Assim:

- Percorremos as peças em ordem crescente. Se a peça a ainda não foi derrubada, então ela deve iniciar uma nova sequência manual.
- Ao iniciar essa nova sequência:
 - incrementamos o tempo global;
 - marcamos a como derrubada nesse instante;
 - definimos $nivel[a] = 0$, pois a é a raiz da nova árvore da BFS;
 - definimos $pai[a] = -1$.
- A propagação a partir de a é feita por BFS. Sempre que uma peça w é descoberta a partir de uma peça v , temos:

$$pai[w] = v \quad \text{e} \quad nivel[w] = nivel[v] + 1.$$

- Como o enunciado define que a propagação ocorre em camadas, o instante em que w cai é dado por:

$$tempo[w] = tempo[a] + nivel[w].$$

Ou seja, peças no mesmo nível caem no mesmo instante.

- Como as raízes das BFSs são escolhidas em ordem crescente e, em cada BFS, os vizinhos também são examinados em ordem crescente, a solução construída é única.
- Para reconstruir o caminho até a peça x , basta seguir os pais de x até a raiz da BFS correspondente e inverter a ordem obtida.
- O instante em que todas as peças terão sido derrubadas é o maior valor armazenado no vetor $tempo$.

```

1 from collections import deque
2
3
4 def reconstruir_caminho(x, pai):
5     caminho = []
6     v = x
7
8     while v != -1:
9         caminho.append(v)
10        v = pai[v]
```

```

11
12     caminho.reverse()
13     return caminho
14
15
16 def solve():
17     n, m = map(int, input().split())
18
19     adj = [[] for _ in range(n + 1)]
20
21     for _ in range(m):
22         u, v = map(int, input().split())
23         adj[u].append(v)
24
25     x = int(input())
26
27     # Apenas para deixar a execucao deterministica
28     for v in range(1, n + 1):
29         adj[v].sort()
30
31     t = 0                                # ordem de visita global
32     instante = 0                         # tempo global do processo
33     F = deque()                          # fila auxiliar da BFS
34     L = [0] * (n + 1)                   # indice de visita (0 = nao visitado)
35     pai = [-1] * (n + 1)                 # pai na floresta de largura
36     nivel = [-1] * (n + 1)               # nivel de cada vertice
37     tempo = [0] * (n + 1)                # instante em que cada peca cai
38
39     for a in range(1, n + 1):             # percorre todas as pecas
40         if L[a] == 0:                     # se a ainda nao foi visitada
41             nivel[a] = 0                  # a sera raiz de uma nova arvore da BFS
42             instante += 1                  # a eh derrubada manualmente neste instante
43             tempo[a] = instante
44             t += 1
45             L[a] = t                      # registra ordem de visita
46             F.append(a)                   # inicia exploracao a partir de a
47
48     maior_tempo_na_bfs = tempo[a]
49
50     while F:                              # enquanto houver vertices na fila
51         v = F.popleft()                   # remove o primeiro elemento da fila
52
53         for w in adj[v]:                   # percorre todos os vizinhos de v
54             if L[w] == 0:                 # w esta sendo descoberto
55                 pai[w] = v
56                 nivel[w] = nivel[v] + 1
57                 tempo[w] = tempo[v] + nivel[w]
58
59                 if tempo[w] > maior_tempo_na_bfs:
60                     maior_tempo_na_bfs = tempo[w]
61
62                 t += 1
63                 L[w] = t
64                 F.append(w)
65
66     # A proxima sequencia manual deve comecar
67     # apos toda a propagacao desta BFS terminar
68     instante = maior_tempo_na_bfs
69
70     # ao terminar esta BFS, o proximo vertice nao visitado
71     # iniciara nova sequencia manual em um novo instante
72
73     caminho_x = reconstruir_caminho(x, pai)
74     tempo_final = max(tempo)
75
76     print(tempo[x])

```

```
76     print(*caminho_x)
77     print(tempo_final)
78
79
80 solve()
```